1 Introducing modern Java

This chapter covers

- Java as a platform and a language
- The new Java release model
- Enhanced Type inference (var)
- Incubating and preview features
- Changing the language
- Small language changes in Java 11

Welcome to Java in 2022. It is an exciting time. Java 17, the latest Long-Term-Support (LTS) release shipped in September 2021, and the first and most adventurous teams are starting to move to it.

At the time of writing, apart from a few trailblazers, Java applications are more or less evenly split between running on Java 11 (released September 2018) and the much older Java 8 (2014). Java 11 offers a lot to recommend, especially for teams that are deploying in the cloud, but some have been a little slow to adopt it.

So, in the first part of this book, we are going to spend some time introducing some of the new features that have arrived in Java 11 and 17. Hopefully, this discussion will help convince some teams and managers who may be reluctant to upgrade from Java 8 that things are better than ever in the newer versions.

Our focus for this chapter is going to be Java 11 because a) it's the LTS version with the largest market share and b) no noticeable adoption of Java 17 has occurred yet. However, in chapter 3, we will introduce the new features in Java 17 to bring you all the way up to date.

Let's get underway by discussing the language-versus-platform duality that lies at the heart of modern Java. This is a critically important point that we'll come back to several times throughout the book, so it's essential to grasp it right at the start.

1.1 The language and the platform

Java as a term can refer to one of several related concepts. In particular, it could mean either the human-readable programming language or the much broader "Java platform."

Surprisingly, different authors sometimes give slightly different definitions of what constitutes a language and a platform. This can lead to a lack of clarity and some confusion about the differences between the two and about which provides the various programming features that application code uses.

Let's make that distinction clear right now, because it cuts to the heart of a lot of the topics in this book. Here are our definitions:

- *The Java language*—The Java language is the statically typed, object-oriented language that we lightly lampooned in the "About this book" section. Hopefully, it's already very familiar to you. One obvious point about source code written in the Java language is that it's human-readable (or it should be!).
- *The Java platform*—The platform is the software that provides a runtime environment. It's the JVM that links and executes your code as provided to it in the form of (not human-readable) class files. It doesn't directly interpret Java language source files but instead requires them to be converted to class files first.

One of the big reasons for the success of Java as a software system is that it's a standard. This means that it has specifications that describe how it's supposed to work. Standardization allows different vendors and project groups to produce implementations that should all, in theory, work the same way. The specs don't make guarantees about how well different implementations will perform when handling the same task, but they can provide assurances about the correctness of the results.

Several separate specs govern the Java system—the most important are the Java Language Specification (JLS) and the JVM Specification (VMSpec). This separation is taken very seriously in modern Java; in fact, the VMSpec no longer makes any reference whatsoever to the JLS directly. We'll have a bit more to say about the differences between these two specs later in the book.

NOTE These days the JVM is actually quite a general-purpose and language-agnostic environment for running programs. This is one reason for the separation of the specs.

One obvious question, when you're faced with the described duality, is, "What's the link between them?" If they're now separate, how do they come together to make the Java

system?

The link between the language and platform is the shared definition of the class file format (the .class files). A serious study of the class file definition will reward you (and we provide one in chapter 4)—in fact, it's one of the ways a good Java programmer can start to become a great one. In figure 1.1, you can see the full process by which Java code is produced and used.

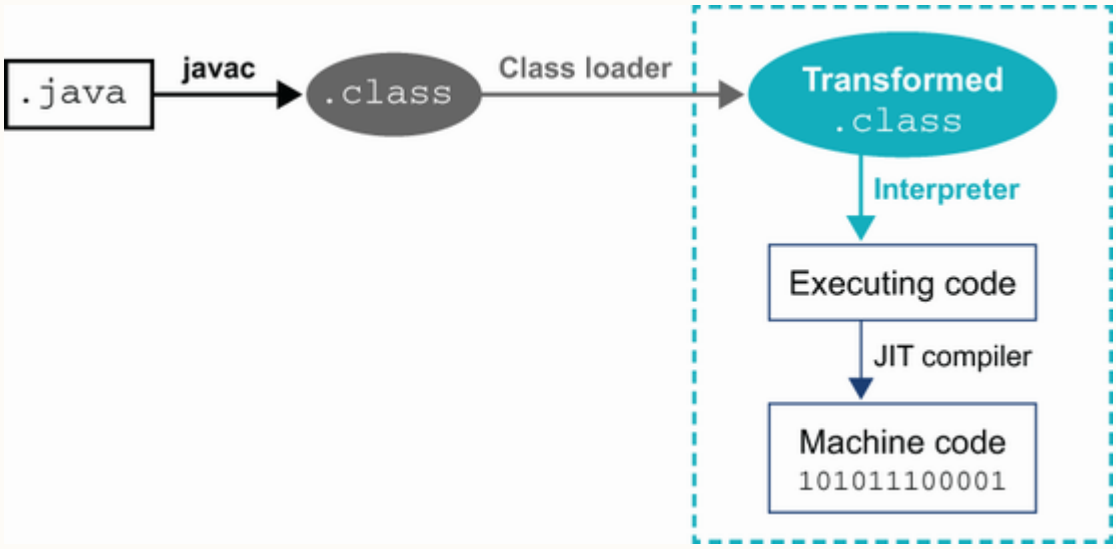


Figure 1.1 Java source code is transformed into .class files, then manipulated at load time before being JIT-compiled.

As you can see in the figure, Java code starts life as human-readable Java source, and it's then compiled by `javac` into a .class file and loaded into a JVM. It's common for classes to be manipulated and altered during the loading process. Many of the most popular Java frameworks transform classes as they're loaded to inject dynamic behavior such as instrumentation or alternative lookups for classes to load.

NOTE Class loading is an essential feature of the Java platform, and we will learn a lot more about it in chapter 4.

Is Java a compiled or interpreted language? The standard picture of Java is of a language that's compiled into .class files before being run on a JVM. If pressed, many developers can also explain that bytecode starts off by being interpreted by the JVM but will undergo just-in-time (JIT) compilation at some later point. Here, however, many people's understanding breaks down into a somewhat hazy conception of bytecode as basically being machine code for an imaginary or simplified CPU.

In fact, JVM bytecode is more like a halfway house between human-readable source and machine code. In the technical terms of compiler theory, bytecode is really a form of intermediate language (IL) rather than actual machine code. This means that the process

of turning Java source into bytecode isn't really compilation in the sense that a C++ or a Go programmer would understand it, and `javac` isn't a compiler in the same sense as `gcc` is—it's really a class file generator for Java source code. The real compiler in the Java ecosystem is the JIT compiler, as you can see in figure 1.1.

Some people describe the Java system as "dynamically compiled." This emphasizes that the compilation that matters is the JIT compilation at runtime, not the creation of the class file during the build process.

NOTE The existence of the source code compiler, `javac`, leads many developers to think of Java as a static, compiled language. One of the big secrets is that at runtime, the Java environment is actually very dynamic—it's just hidden a bit below the surface.

So, the real answer to "Is Java compiled or interpreted?" is "both." With the distinction between language and platform now clearer, let's move on to talk about the new Java release model.

1.2 The new Java release model

Java was not always an open source language, but following an announcement at the JavaOne conference in 2006, the source code for Java itself (minus a few bits that Sun didn't own the source for) was released under the GPLv2+CE license (**https://openjdk.java.net/legal/gplv2+ce.html**).

This was around the time of the release of Java 6, so Java 7 was the first version of Java to be developed under an open source software (OSS) license. The primary focus for open source development of the Java platform since then has been the OpenJDK project (**https://openjdk.java.net**), and that continues to this day.

A lot of the project discussion takes place on mailing lists that cover aspects of the overall codebase. There are "permanent" lists such as core-libs (core libraries), as well as more transient lists that are formed as part of specific OpenJDK projects such as lambda-dev (lambdas), which then become inactive when a particular project has been completed. In general, these lists have been the relevant forums for discussing possible future features, allowing developers from the wider community to participate in the process of producing new versions of Java.

NOTE Sun Microsystems was acquired by Oracle shortly before Java 7 was released. Therefore, all of Oracle's releases of Java have been based on the open source codebase.

The open source releases of Java had settled into a feature-driven release cycle, where a single marquee feature effectively defines the release (e.g., lambdas in Java 8 or modules in Java 9).

With the release of Java 9, however, the release model changed. From Java 10 onward, Oracle decided that Java would be released on a strict, time-based model. This means that OpenJDK now uses a *mainline* development model, which includes the following:

- New features are developed on a branch and merged only when they are code complete.
- Releases can occur on a strict time cadence.
- Late features do not delay releases but are held over for the next release.
- The current head of the trunk should always be releasable (in theory).
- If necessary, an emergency fix can be prepared and pushed out at any point.
- Separate OpenJDK projects are used to explore and research longer-term, future directions.

A new version of Java is released every six months ("feature releases"). The various providers (Oracle, Eclipse Adoptium, Amazon, Azul, et al.) can choose to make *any* of those releases a Long-Term Support (LTS) release. However, in practice, all of the vendors follow having one release every three years being named as the LTS release.

NOTE As of late 2021, discussions are underway to reduce the LTS gap from three years to two years. We may well see the next LTS version as Java 21 in 2023 as opposed to Java 23 in 2024.

The first LTS release was Java 11, with Java 8 retrospectively included in the set of LTS releases. Oracle's intention was for the Java community to upgrade regularly and to take up the feature releases as they emerge. However, in practice, the community (and enterprise customers in particular) have proved to be resistant to this model, preferring instead to upgrade from one LTS release to the next.

This approach, of course, limits the uptake of new Java features and stifles innovation. However, the realities of enterprise software are what they are, and many people still view an upgrade of the Java version as a significant undertaking.

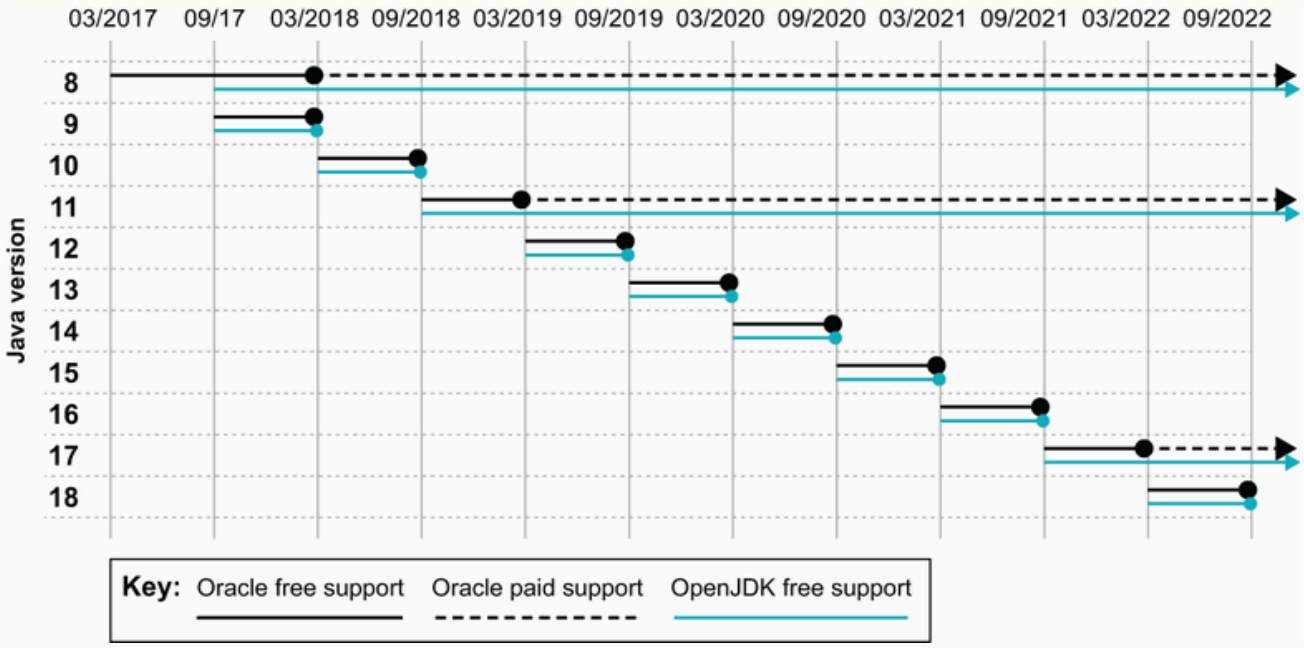


Figure 1.2 The timescale of recent and future releases

This means that whereas the release road map shown in figure 1.2 contains a major release every six months, the only releases that have significant usage are the LTS versions—Java 17 (which was just released in September 2021), Java 11 (which was released in September 2018), and the pre-modules release, Java 8, which is more than seven years old. Java 8 and Java 11 have roughly equal market share, with Java 11 recently having taken over 50% and rapidly accelerating. Java 17 adoption is expected to be much quicker than the move from Java 8 to Java 11 because the most difficult hurdles introduced by the module system and security restrictions will have already been overcome with the earlier migration.

The other significant change in the new release model is that Oracle has changed the license for their distribution. Although Oracle's JDK is built from the OpenJDK sources, the binary is not licensed under an OSS license. Instead, Oracle's JDK is proprietary software, and as of JDK 11, Oracle provides support and updates for only six months for each version. This means that many people who relied on Oracle's free updates are now faced with a choice:

- Pay Oracle for support and updates, or
- Use a different distribution that produces open source binaries.

Alternative JDK vendors include Eclipse Adoptium (previously AdoptOpenJDK), Alibaba (Dragonwell), Amazon (Corretto), Azul Systems (Zulu), IBM, Microsoft, Red Hat, and SAP.

NOTE Two of the authors (Martijn and Ben) helped found the AdoptOpenJDK project, which has evolved into the vendor-neutral Eclipse Adoptium community project to build

and release a high-quality, free, and open source Java binary distribution. See **adoptium.net** for more details.

With the licensing changes and with so many providers, picking the correct Java for you and your team is a choice that you should make with care. Thankfully, leaders in the Java ecosystem have written some very detailed guides, and appendix A distills them down for you.

Although the Java release model has changed to use timed releases, the vast majority of teams are still running on either JDK 8 or 11. These LTS releases are being maintained by the community (including major vendors) and still receive regular security updates and bug fixes. The changes made to the LTS versions are deliberately small in scope and are "housekeeping updates." Apart from security and small bug fixes, only a minimal set of changes are permitted. These include fixes needed to ensure that the LTS releases will continue to work correctly for their expected lifetime. This includes things like the following:

- The addition of the new Japanese Era
- Time zone database updates
- TLS 1.3
- Adding Shenandoah, a low-pause GC for large modern workloads

One other necessary change is that the build scripts for macOS needed to be updated to work with a recent version of Apple's Xcode tool so that they will continue to work on new releases of Apple's operating system.

Within the projects to maintain JDK 8 and 11 (sometimes called the "updates" projects), some potential scope still exists for new features to be backported, but it is minimal. As an example, one of the guiding rules is that newly ported features may not change program semantics. Examples of permissible changes could include the support for TLS 1.3 or the backport of Java Flight Recorder to Java 8u272.

Now that we've set the scene by clarifying the difference between the language and platform and explaining the new release model, let's meet our first technical feature of modern Java. The new feature we're going to meet is something that developers have been asking for since almost the first release of Java—a way to reduce the amount of typing that writing Java programs seems to involve.

1.3 Enhanced type inference (var keyword)

Java has historically had a reputation as a verbose language. However, in recent versions, the language has evolved to make more and more use of *type inference*. This feature of the source code compiler enables the compiler to work out some of the type information in programs automatically. As a result, it doesn't need to be told everything explicitly.

NOTE The aim of type inference is to reduce boilerplate content, remove duplication, and allow for more concise and readable code.

This trend started with Java 5, when generic methods were introduced. Generic methods permit a very limited form of type inference of generic type arguments, so that instead of having to explicitly provide the exact type that is needed, like this:

```
List<Integer> empty = Collections.<Integer>emptyList();
```

the generic type parameter can be omitted on the right-hand side, like so:

```
List<Integer> empty = Collections.emptyList();
```

This way of writing a call to a generic method is so familiar that many developers will struggle to remember the form with explicit type arguments. This is a good thing—it means the type inference is doing its job and removing the superfluous boilerplate content so that the meaning of the code is clear.

The next significant enhancement to type inference in Java came with version 7, which introduced a change when dealing with generics. Before Java 7, it was common to see code like this:

```
Map<Integer, Map<String, String>> usersLists =
                    new HashMap<Integer, Map<String, String>>();
```

That is a really verbose way to declare that you have some users, whom you identify by `userid` (which is an integer), and each user has a set of properties (modeled as a map of string to strings) specific to that user.

In fact, almost half of the source is duplicated characters, and they don't tell us anything. So, from Java 7 onward, we can write

```
Map<Integer, Map<String, String>> usersLists = new HashMap<>();
```

and have the compiler work out the type information on the right side. The compiler is working out the correct type for the expression on the right side— it isn't just substituting the text that defines the full type.

NOTE Because the shortened type declaration looks like a diamond, this form is called "diamond syntax."

In Java 8, more type inference was added to support the introduction of lambda expressions, like this example where the type inference algorithm can conclude that the type of `s` is a `String`:

```
Function<String, Integer> lengthFn = s -> s.length();
```

In modern Java, type inference has been taken one step further, with the arrival of *Local Variable Type Inference* (LVTI), otherwise known as `var`. This feature was added in Java 10 and allows the developer to infer the types of *variables*, instead of the types of *values*, like this:

```
var names = new ArrayList<String>();
```

This is implemented by making `var` a reserved, "magic" type name rather than a language keyword. Developers can still in theory use `var` as the name of a variable, method, or package.

NOTE An important side effect of using `var` appropriately is that the domain of your code is once more front and center (as opposed to the type information). But with great power comes great responsibility! Make sure that you name your variables carefully to help future readers of your code.

On the other hand, code that previously used `var` as the name of a type will have to be recompiled. However, virtually all Java developers follow the convention that type names should start with a capital letter, so the number of instances of preexisting types

called `var` should be vanishing small. This means that it is entirely legal to write code like that shown in the next listing.

Listing 1.1 Bad code

```
package var;

public class Var {
  private static Var var = null;

  public static Var var() {
    return var;
  }

  public static void var(Var var) {
    Var.var = var;
  }
}
```

And then call it like this:

```
var var = var();
if (var == null) {
  var(new Var());
}
```

However, just because something is *legal*, does not mean it is *sensible*. Writing code like the previous listing is not going to make you any friends and should not pass code reviews!

The intention of `var` is to reduce verbosity in Java code and to be familiar to programmers coming to Java from other languages. It does not introduce dynamic typing, and all Java variables continue to have static types at all times—you just don't need to write them down explicitly in all cases.

Type inference in Java is *local*, and in the case of `var`, the algorithm examines only the declaration of the local variable. This means it cannot be used for fields, method arguments, or return types. The compiler applies a form of *constraint solving* to determine whether any type exists that could satisfy all the requirements of the code as written.

NOTE `var` is implemented solely in the source code compiler (`javac`) and has no run-time or performance effect whatsoever.

For example, in the declaration of `lengthFn` in the previous code sample, the constraint solver can deduce that the type of the method parameter `s` must be compatible with `String` which is explicitly provided as the type of the parameter to `Function`. In Java, of course, the string type is `final`, so the compiler can conclude that the type of `s` is exactly `String`.

For the compiler to be able to infer types, enough information must be provided by the programmer to allow the constraint equations to be solved. For example, code like this

```
var fn = s -> s.length();
```

does not have enough type information for the compiler to deduce the type of `fn`, and so it will not compile. One important case of this is

```
var n = null;
```

which cannot be resolved by the compiler because the null value can be assigned to a variable of any reference type, so there is no information about what types `n` could conceivably be. We say that the type constraint equations that the inferencer needs to solve are "underdetermined" in this case—a mathematical term that connects the number of equations to be solved with the number of variables.

You could imagine a scheme of type inference that goes beyond just the initial declaration of the local variable and examines more code to make inference decisions, like this:

```
var n = null;
String.format(n);
```

A more complex inference algorithm (or a human) might be able to conclude that the type of `n` is actually `String`, because the `format()` method takes a string as the first argument.

This might seem appealing, but, as with everything else in software, it represents a trade-off. More complexity means longer compilation times and a wider variety of ways

in which the inference can fail. This, in turn, means that the programmer must develop a more complicated intuition to use nonlocal type inference correctly.

Other languages may choose to make different trade-offs, but Java is clear: only the declaration is used to infer types. Local variable type inference is intended to be a beneficial technique to reduce boilerplate text and verbosity. However, it should be used only where necessary to make the code clearer, not as a blunt instrument to be used whenever possible (the "Golden Hammer" antipattern).

Some quick guidelines for when to use LVTI follow:

- In simple initializers, if the right-hand side is a call to a constructor or static factory method
- If removing the explicit type deletes repeated or redundant information
- If variables have names that already indicate their types
- If the scope and usage of the local variable is short and simple

A complete set of applicable rules of thumb is provided by Stuart Marks, one of the core developers of the Java language, in his style guides for LVTI usage at **http://mng.bz/RvPK**.

To conclude this section, let's look at another, more advanced, usage of `var`—the so-called *nondenotable types*. These are types that are legal in Java, but they cannot appear as the type of a variable. Instead, they must be inferred as the type of the expression that is being assigned. Let's look at a simple example using the `jshell` interactive environment, which arrived in Java 9:

```
jshell> var duck = new Object() {
   ...>     void quack() {
   ...>         System.out.println("Quack!");
   ...>     }
   ...> }
duck ==> $0@5910e440

jshell> duck.quack();
Quack!
```

The variable `duck` has an unusual type—it is effectively `Object` but extended with a method called `quack()`. Although the object may quack like a duck, its type lacks a name, so we can't use the type as either a method parameter or return type.

With LVTI, we can use it as the inferred type of a local variable. This allows us to use the type within a method. Of course, the type can't be used outside of this tight local scope, so the overall utility of this language feature is limited. It's more of a curiosity than anything else.

Despite these limitations, this does represent a glimpse at Java's take on a feature that is present in some other languages—sometimes referred to as *structural typing* in statically typed languages and *duck typing* in dynamically typed languages (particularly Python).

1.4 Changing the language and the platform

We think it's essential to explain the "why" of language change as well as the "what." During the development of new versions of Java, much interest around new language features often exists, but the community doesn't always understand how much work is required to get changes fully engineered and ready for prime time.

You may also have noticed that in a mature runtime such as Java, language features tend to evolve from other languages or libraries, make their way into popular frameworks, and only then get added to the language or runtime itself. We hope to shed a bit of light on this area and hopefully dispel a few myths along the way. But if you're not very interested in how Java evolves, feel free to skip ahead to section 1.5 and jump right into the language changes.

There is an effort curve involved in changing the Java language—some possible implementations require less engineering effort than others. In figure 1.3, we've tried to represent the different routes and show the relative effort required for each.

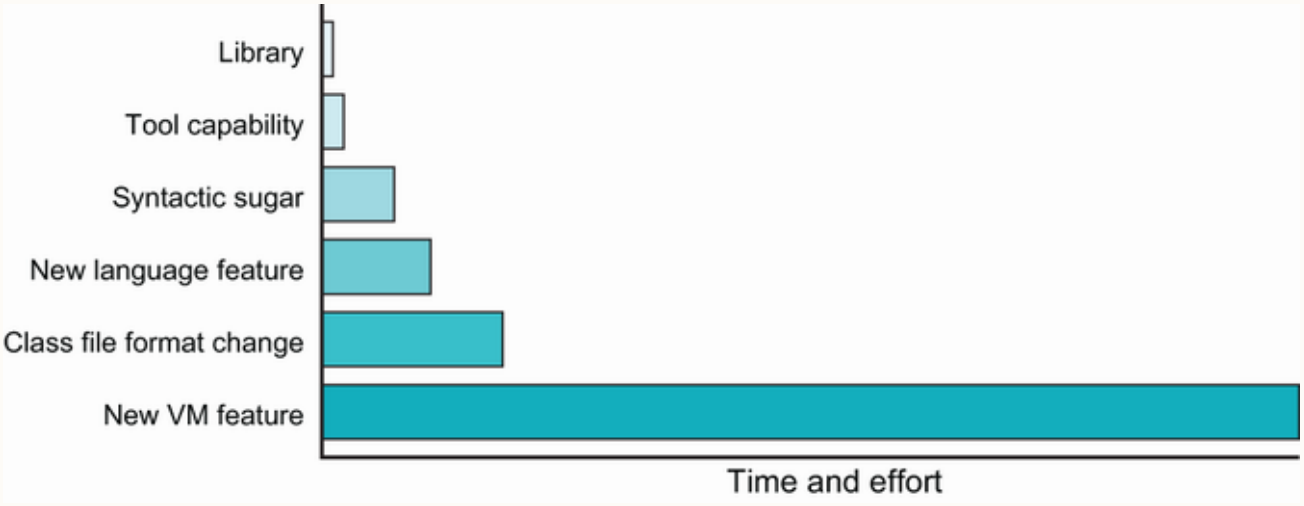


Figure 1.3 The relative effort involved in implementing new functionality in different ways

In general, it's better to take the route that requires the least effort. This means that if it's possible to implement a new feature as a library, you generally should. But not all features are easy, or even possible, to implement in a library or an IDE capability. Some features have to be implemented deeper inside the platform. Here's how some recent features fit into our complexity scale for new language features:

- *Library change*—Collections factory methods (Java 9)
- *Syntactic sugar*—Underscores in numbers (Java 7)
- *Small new language feature*—try-with-resources (Java 7)
- *Class file format change*—Annotations (Java 5)
- *New JVM feature*—Nestmates (Java 11)
- *Major new feature*—Lambda Expressions (Java 8)

Let's take a close look at how changes across the complexity scale are made.

4.1 Sprinkling some sugar

A phrase that's sometimes used to describe a language feature is "syntactic sugar." That is, the syntactic sugar form is provided because it's easier for humans to work with despite the functionality already existing in the language.

As a rule of thumb, a feature referred to as syntactic sugar is removed from the compiler's representation of the program early in the compilation process—it's said to have been "desugared" into the basic representation of the same feature.

This makes syntactic sugar changes to a language easier to implement because they usually involve a relatively small amount of work and only involve changes to the compiler (`javac` in the case of Java).

One question that might well be asked at this point is, "What constitutes a small change to the spec?" One of the most straightforward changes in Java 7 consisted of adding a single word—"String"—to section 14.11 of the JLS, which allowed strings in a switch statement. You can't really get much smaller than that as a change, and yet even this change touches several other aspects of the spec. Any alteration produces consequences, and these have to be chased through the entire design of the language.

4.2 Changing the language

The full set of actions that must be performed (or at least investigated) for *any* change follows:

- Update the JLS.
- Implement a prototype in the source compiler.
- Add library support essential for the change.
- Write tests and examples.
- Update documentation.

In addition, if the change touches the JVM or platform aspects, the following actions must occur:

- Update the VMSpec.
- Implement the JVM changes.
- Add support in the class file and JVM tools.
- Consider the impact on reflection.
- Consider the impact on serialization.
- Think about any effects on native code components, such as Java Native Interface (JNI).

This isn't a small amount of work, and that's after the impact of the change across the whole language spec has been considered!

An area of hairiness, when it comes to making changes, is the type system. That isn't because Java's type system is terrible. Instead, languages with rich static type systems are likely to have a lot of possible interaction points between different bits of those type systems. Making changes to them is prone to creating unexpected surprises.

4.3 JSRs and JEPs

Two main mechanisms are used to make changes to the Java platform. The first is the *Java Specification Request* (JSR), which is specified by the *Java Community Process* (JCP). This is used to determine standard APIs—both external libraries and major internal platform APIs.

This was historically the only way of making changes to the Java platform and was best used to codify a consensus of already mature technology. However, in recent years, a desire to implement change faster (and in smaller units) led to the development of the *JDK Enhancement Proposal* (JEP) as a lighter-weight alternative. Platform (aka umbrella) JSRs are now made up of JEPs targeted for the next version of Java. The JSR process is used to grant extra intellectual property protections for the whole ecosystem.

When discussing new Java features, it is often useful to refer to an upcoming or recent feature by its JEP number. A complete list of all JEPs, including those that have been de-

livered or withdrawn, can be found at **https://openjdk.java.net/jeps/0**.

4.4 Incubating and preview features

Within the new release model, Java has two mechanisms for trying out a proposed feature before finalizing it in a later release. The aim of these mechanisms is to provide better features by gathering feedback from a much wider pool of users and potentially changing or withdrawing the feature before it becomes a permanent part of Java.

Incubating features are new APIs and their implementation, which in their simplest form are effectively just a new API shipped as a self-contained module (we will meet the details of Java modules in chapter 2). The name of the module is chosen so that it makes it clear that the API is temporary and will change when the feature is finalized.

NOTE This means that any code that relies upon a nonfinalized version of an incubating feature will have to make changes when the feature becomes final.

One very visible example of an incubating feature is the new support for version 2 of the HTTP protocol, usually referred to as HTTP/2. In Java 9, this was shipped as the incubator module `jdk.incubator.http`. The naming of this module, and the use of the `jdk.incubator` namespace rather than `java` clearly marked the feature as nonstandard and subject to change. The feature was standardized in Java 11 when it was moved to the `java.net.http` module in the `java` part of the namespace.

NOTE We will meet another incubating feature in chapter 18 when we discuss the Foreign Access API, which is part of an OpenJDK project codenamed Panama.

The main advantage of this approach is that an incubating feature can be isolated to a single namespace. Developers can quickly try out the feature and even use it in production code, providing they are happy to modify some code and recompile and relink when the feature becomes standardized.

Preview features are the other mechanism that recent Java versions provide for shipping nonfinalized features. They are more intrusive than incubating features because they are implemented as part of the language itself, at a deeper level. These features potentially require support from the following:

- The `javac` compiler
- Bytecode format
- Class file and class loading

They are available only if specific flags are passed to the compiler and runtime. Trying to use preview features without the flags enabled is an error, both at compile time and at runtime.

This makes them much more complex to handle (compared to incubating features). As a result, preview features can't really be used in production. For one thing they are represented by a version of the classfile format that is not finalized and may never be supported by any production version of Java.

This means that preview features are suitable only for experimentation, developer testing, and familiarization. Unfortunately, in almost all deployments, only fully finalized features can be used in code that is destined for production.

Java 11 did not contain any preview features (although a first preview version of *switch expressions* arrived in Java 12), so it's hard to give a good example of one in this section. We'll dig more into preview versions in chapter 3 when we discuss Java 17, though.

1.5 Small changes in Java 11

Since Java 8, a relatively large number of new small features have appeared in successive releases. Let's take a quick tour through some of the most important ones—although this is by no means all the changes.

5.1 Collections factories (JEP 213)

An often-requested enhancement is to extend Java to support a simple way to declare *collection literals*—a dumb collection of objects (such as a list or a map). This seems attractive because many other languages support some form of this, and Java itself has always had array literals, as shown here:

```
jshell> int[] numbers = {1, 2, 3};
numbers ==> int[3] { 1, 2, 3 }
```

However, although it seems superficially attractive, adding this feature at the language level has some significant drawbacks. For example, although `ArrayList`, `HashMap`, and `HashSet` are the implementations that are most familiar to developers, a primary design principle of the Java Collections are that they are represented as interfaces, not classes. Other implementations are available and are widely used.

This means that it would run counter to the design intent to have a new syntax that directly couples to specific implementations, no matter how common. Instead, the design decision was to add simple factory methods to the relevant interfaces, exploiting the fact that Java 8 added the ability to have static methods on interfaces. The resulting code looks like this:

```
Set<String> set = Set.of("a", "b", "c");

var list = List.of("x", "y");
```

Although this method is a little more verbose than adding support at language level, the complexity cost in implementation terms is substantially less. These new methods are implemented as a set of overloads as follows:

```
List<E> List<E>.<E>of()
List<E> List<E>.<E>of(E e1)
List<E> List<E>.<E>of(E e1, E e2)
List<E> List<E>.<E>of(E e1, E e2, E e3)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9,
  E e10)
List<E> List<E>.<E>of(E... elements)
```

The common cases (up to 10 elements) are provided, along with a varargs form for the unlikely use case that more than 10 elements are required in the collection.

For maps, the situation is a little more complicated, because maps have two generic parameters (the key type and the value type) and so, although the simple cases can be written like this:

```
var m1 = Map.of(k1, v1);
var m2 = Map.of(k1, v1, k2, v2);
```

there is no simple way of writing the equivalent of the varargs form for map. Instead, a different factory method, `ofEntries()`, is used in combination with a static helper

method, `entry()`, to provide an equivalent of a varargs form, as shown next:

```
Map.ofEntries(
   entry(k1, v1),
   entry(k2, v2),
   // ...
   entry(kn, vn));
```

One final point that developers should be aware of: the factory methods produce instances of immutable types, as follows:

```
jshell> var ints = List.of(2, 3, 5, 7);
ints ==> [2, 3, 5, 7]

jshell> ints.getClass();
$2 ==> class java.util.ImmutableCollections$ListN
```

These class are new implementations of the Java Collections interfaces that are immutable—they are not the familiar, mutable classes (such as `ArrayList` and `HashMap`). Attempts to modify instances of these types will result in an exception being thrown.

5.2 Remove enterprise modules (JEP 320)

Over time, Java Standard Edition (aka Java SE) had a few modules added to it that were really part of Java Enterprise Edition (Java EE) such as

- JAXB
- JAX-WS
- CORBA
- JTA

In Java 9, the following packages that implemented these technologies were moved into noncore modules and deprecated for removal:

- java.activation (JAF)
- java.corba (CORBA)
- java.transaction (JTA)
- java.xml.bind (JAXB)
- java.xml.ws (JAX-WS, plus some related technologies)
- java.xml.ws.annotation (Common Annotations)

As part of an effort to streamline the platform, in Java 11 these modules have been removed. The following three related modules used for tooling and aggregation have also been removed from the core SE distribution:

- java.se.ee (aggregator module for the six modules above)
- jdk.xml.ws (tools for JAX-WS)
- jdk.xml.bind (tools for JAXB)

Projects built on Java 11 and later that want to use these capabilities now require the inclusion of an explicit external dependency. This means that some programs that relied upon these APIs built cleanly under Java 8 but require modifications to their build script to build under Java 11. We will investigate this specific issue more fully in chapter 11.

5.3 HTTP/2 (Java 11)

In modern times, a new version of the HTTP standard has been released—HTTP/2. We're going to examine the reasons for finally updating the venerable HTTP 1.1 specification (dating from 1997!). Then we'll see how Java 11 gives the well-grounded developer access to the new features and performance of HTTP/2.

As you might expect for technology from 1997, HTTP 1.1 has been showing its age, particularly around performance in modern web applications. Limitations include problems such as:

- Head-of-line blocking
- Restricted connections to a single site
- Performance overhead of HTTP control headers

HTTP/2 is a *transport*-level update to the protocol focused on fixing these sorts of fundamental performance issues that don't fit how the web really works today. With its performance focus on how bytes flow between client and server, HTTP/2 actually doesn't alter many of the familiar HTTP concepts—request/response, headers, status codes, response bodies—all of these remain semantically the same in HTTP/2 vs. HTTP 1.1.

Head-of-line blocking

Communication in HTTP takes place over TCP sockets. Although HTTP 1.1 defaulted to reusing individual sockets to avoid repeating unnecessary setup costs, the protocol dictated that requests be returned in order, even when multiple requests shared a socket (known as pipelining; see figure 1.4). This means that a slow response from the server blocked subsequent requests, which theoretically could have been returned sooner.

These effects are readily visible in places like browser rendering stalling on downloading assets. The same one-response-per-connection-at-a-time behavior can also limit JVM applications talking to HTTP-based services.

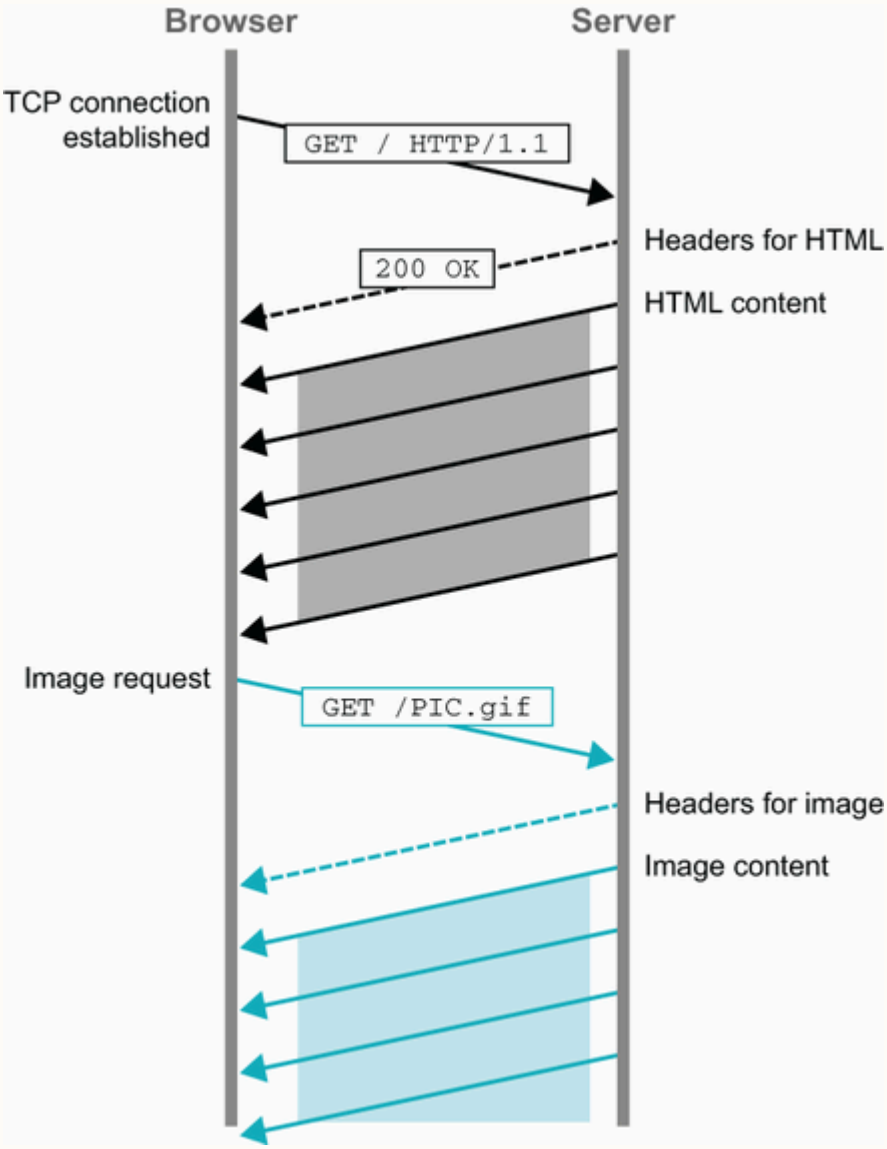


Figure 1.4 HTTP 1.1 transfers

HTTP/2 is designed from the ground up to multiplex requests over the same connection, as shown in figure 1.5. Multiple *streams* between the client and server are always supported. It even allows for separately receiving the headers and the body of a single request.

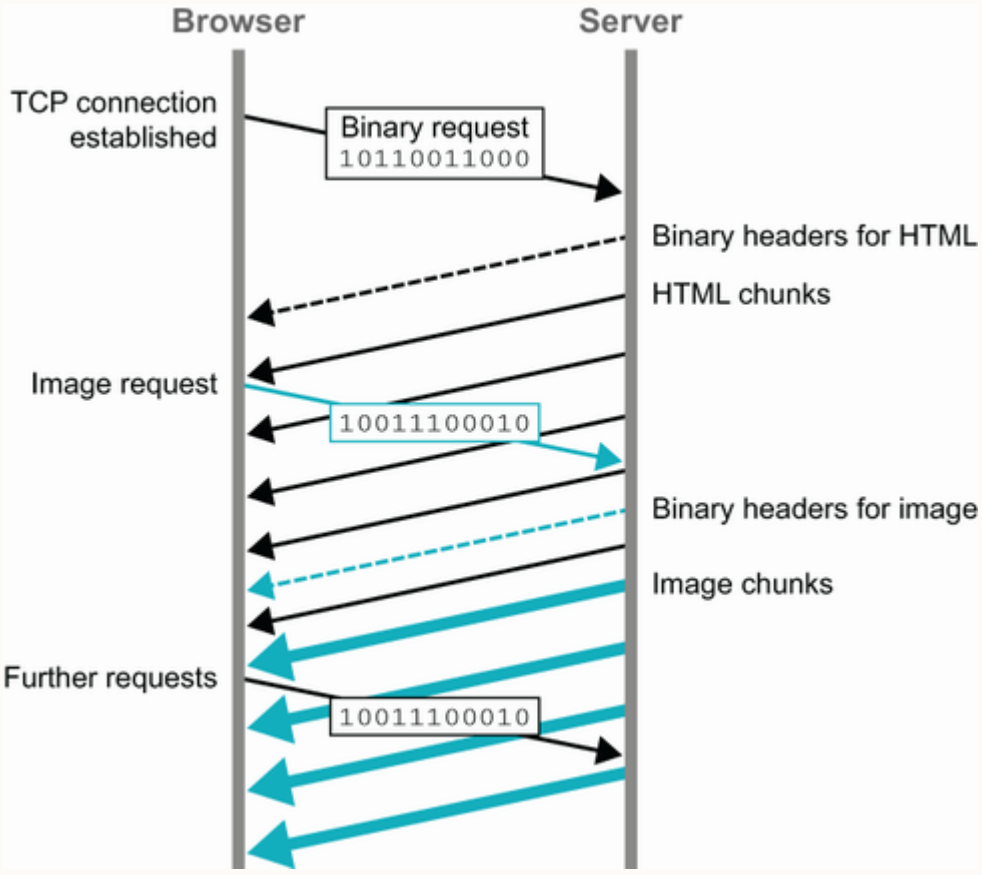


Figure 1.5 HTTP/2 transfers

This fundamentally changes assumptions that decades of HTTP 1.1 have made second nature to many developers. For instance, it's long been accepted that returning lots of small assets on a website performed worse than making larger bundles. JavaScript, CSS, and images all have common techniques and tooling for smashing many smaller files together to return more efficiently. In HTTP/2, multiplexed responses mean your resources don't get blocked behind other slow requests, and smaller responses may be more accurately cached, yielding a better experience overall.

Restricted connections

The HTTP 1.1 specification recommends limiting to two connections to a server at a time. This is listed as a *should* rather than a *must*, and modern web browsers often allow between six and eight connections per domain. This limit to concurrent downloads from a site has often led developers to serve sites from multiple domains or implement the sort of bundling mentioned before.

HTTP/2 addresses this situation: each connection can effectively be used to make as many simultaneous requests as desired. Browsers open only one connection to a given domain but can perform many requests over that same connection at the same time.

In our JVM applications, where we might have pooled HTTP 1.1 connections to allow for more concurrent activity, HTTP/2 gives us another built-in way to squeeze out more

requests.

HTTP header performance

A significant feature of HTTP is the ability to send *headers* alongside requests. Headers are a critical part of how the HTTP protocol itself is stateless, but our applications can maintain state between requests (such as the fact your user is logged in).

Although the body of HTTP 1.1 payloads may be compressed if the client and server can agree on the algorithm (typically gzip), headers don't participate. As richer web applications make more and more requests, the repetition of increasingly large headers can be a problem, especially for larger websites.

HTTP/2 addresses this problem with a new binary format for headers. As a user of the protocol, you don't have to think much about this—it's simply built in to how headers are transmitted between client and server.

TLS all the things

In 1997, HTTP 1.1 entered a very different internet than we see today. Commerce on the internet was only starting to take off, and security wasn't always a top concern in early protocol designs. Computing systems were also slow enough to make practices like encryption often far too expensive.

HTTP/2 was officially accepted in 2015 into a world that was far more security conscious. In addition, the computing needs for ubiquitous encryption of web requests through TLS (known in earlier versions as SSL) are low enough to have removed most arguments over whether or not to encrypt. As such, in practice, HTTP/2 is supported only with TLS encryption (the protocol does, in theory, allow for transmission in cleartext, but none of the major implementations provide it).

This has an operational impact on deploying HTTP/2, because it requires a certificate with a lifecycle of expiration and renewal. For enterprises, this increases the need for certificate management. Let's Encrypt (**https://www.letsencrypt.org**), and other private options have been growing in response to this need.

Other considerations

Although the future is trending toward the uptake of HTTP/2, deployment of it across the web hasn't been fast. In addition to the encryption requirement, which even impacts

local development, this delay may be attributable to the following rough edges and extra complexity:

- HTTP/2 is binary-only; working with an opaque format is challenging.
- HTTP layer products such as load balancers, firewalls, and debugging tools require updates to support HTTP/2.
- Performance benefits are aimed mainly at the browser-based use of HTTP. Backend services working over HTTP may see less benefit to updating.

HTTP/2 in Java 11

The arrival of a new HTTP version after so many years motivated JEP 110 to introduce an entirely new API. Within the JDK, this replaces (but doesn't remove) `HttpURLConnection` while aiming to put a usable HTTP API "in the box," as it were, because many developers have reached for external libraries to fulfill their HTTP-related needs.

The resulting HTTP/2- and web socket–compatible API came first to Java 9 as an Incubating feature. JEP 321 moved it to its permanent home in Java 11 under `java.net.http`. The new API supports HTTP 1.1 as well as HTTP/2 and can fall back to HTTP 1.1 when a server being called doesn't support HTTP/2.

Interactions with the new API start from the `HttpRequest` and `HttpClient` types. These are instantiated via builders, setting configurations before issuing the actual HTTP call, as shown next:

```
var client = HttpClient.newBuilder().build();          ❶

var uri = new URI("https://google.com");
var request = HttpRequest.newBuilder(uri).build();     ❷

var response = client.send(                            ❸
   request,
   HttpResponse.BodyHandlers.ofString(                 ❹
      Charset.defaultCharset()));

System.out.println(response.body());
```

❶ Constructs an HttpClient instance we can use to make requests

❷ Constructs a specific request to Google with an HttpRequest instance

❸ Synchronously makes the HTTP request and saves its response. This line blocks until the entire request has completed.

❹ The send method needs a handler to tell it what to do with the response body. Here we use a standard handler to return the body as a String.

This demonstrates the synchronous use of the API. After building our request and client, we issue the HTTP call with the `send` method. We won't receive the `response` object back until the full HTTP call has completed, much like the older HTTP APIs in the JDK.

The first parameter is the request we set up, but the second deserves a closer look. Rather than expecting to always return a single type, the `send` method expects us to provide an implementation of the `HttpResponse.BodyHandler<T>` interface to tell it how to handle the response. `HttpResponse.BodyHandlers` provides some useful basic handlers for receiving your response as a byte array, as a string, or as a file. But customizing this behavior is just an implementation of `BodyHandler` away. All of this plumbing is based on the `java.util.concurrent.Flow` publisher and subscriber mechanisms, a form of programming known as reactive streams.

One of the most significant benefits of HTTP/2 is its built-in multiplexing. Only using a synchronous `send` doesn't really gain those benefits, so it should come as no surprise that `HttpClient` also supports a `sendAsync` method. `sendAsync` returns a `CompletableFuture` wrapped around the `HttpResponse`, providing a rich set of capabilities that may be familiar from other parts of the platform, as shown here:

```
var client = HttpClient.newBuilder().build();

var uri = new URI("https://google.com");
var request = HttpRequest.newBuilder(uri).build();          ❶

var handler = HttpResponse.BodyHandlers.ofString();
CompletableFuture.allOf(                                    ❷
   client.sendAsync(request, handler)                       ❸
      .thenAccept((resp) ->                                 ❹
            System.out.println(resp.body()),
   client.sendAsync(request, handler)                       ❺
      .thenAccept((resp) ->                                 ❺
            System.out.println(resp.body()),                ❺
   client.sendAsync(request, handler)                       ❺
      .thenAccept((resp) ->
            System.out.println(resp.body())
).join();
```

❶ Creates the client and request as before

❷ Uses CompletableFuture.allOf to wait for all the requests to finish

❸ sendAsync starts an HTTP request but returns a future and does not block.

❹ When the future completes, we use thenAccept to receive the response.

❺ We can reuse the same client to make multiple requests simultaneously.

Here we set up a request and client again, but then we asynchronously repeat the call three separate times. `CompletableFuture.allOf` combines these three futures, so we can wait on them all to finish with a single `join`.

This only scratches the two main entry points to this API. It offers tons of features and customization, from the configuration of timeouts and TLS, all the way to advanced asynchronous features like receiving HTTP/2 server pushes via `HttpResponse.PushPromiseHandler`.

Building off the futures and reactive streams, the new HTTP API in the JDK provides an attractive alternative to the large libraries that have dominated the ecosystem in the HTTP space. Designed with modern asynchronous programming at the forefront, `java.net.http` puts Java in an excellent place for wherever the web evolves to in the future.

5.4 Single-file source-code programs (JEP 330)

The usual way that Java programs are executed is by compiling source code to a class file and then starting up a virtual machine process that acts as an execution container to interpret the bytecode of the class.

This is very different from languages like Python, Ruby, and Perl, where the source code of a program is interpreted directly. The Unix environment has a long history of these types of *scripting languages*, but Java has not traditionally been counted among them.

With the arrival of JEP 330, Java 11 offers a new way to execute programs. Source code can be compiled in memory and then executed by the interpreter without ever producing a .class file on disk, as shown in figure 1.6.

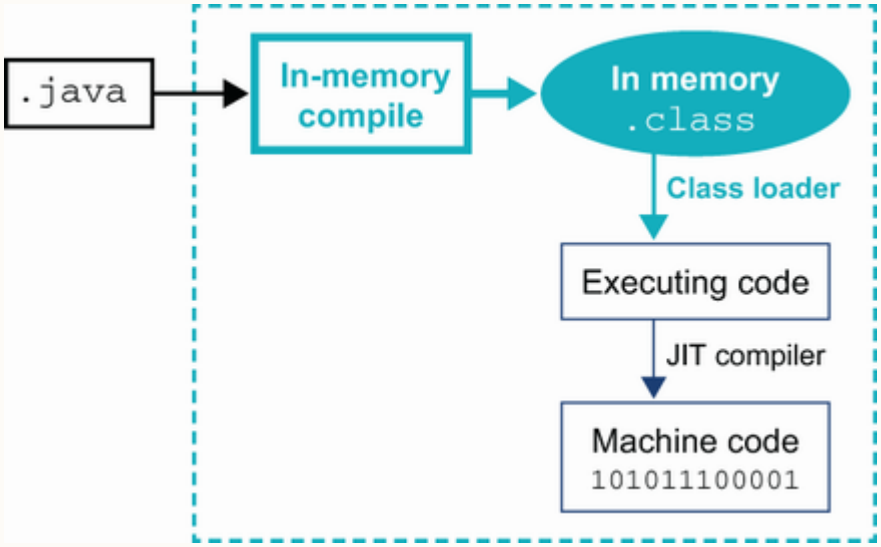


Figure 1.6 Single file execution

This gives a user experience that is like Python and other scripting languages.

The feature has some limitations, including the following:

- It is limited to code that lives in a single source file.
- It cannot compile additional source files in the same run.
- It may contain any number of classes in the source file.
- It must have the first class declared in the source file as the entry point.
- It must define the main method in the entry point class.

The feature also uses a `--source` flag to indicate source code compatibility mode—essentially the language level of the script.

Java file-naming conventions must be followed for execution, so the class name should match the filename. However, the .java extension should *not* be used because this can confuse the launcher.

These types of Java scripts can also contain a shebang line, as shown next:

```
#!/usr/bin/java --source 11

public final class HTTP2Check {
   public static void main(String[] args) {
      if (args.length < 1) {
         usage();
      }
      // implementation of our HTTP callers...      (1)
```

```
    }
}
```

❶ Full code for HTTP2Check is provided in project resources.

The shebang line provides the necessary parameters so that the file can be marked executable and directly invoked, like this:

```
$ ./HTTP2Check https://www.google.com
https://www.google.com: HTTP_2
```

Although this feature does not bring the full experience of scripting languages to Java, it can be a useful way of writing simple, useful tools in the Unix tradition without introducing another programming language into the mix.

Summary

- The Java language and platform are two separate (if strongly related) components of the Java ecosystem. The platform supports many languages beyond just Java.
- After Java 8, the Java platform has adopted a new timed-release process. New versions arrive every six months and a Long-Term-Support (LTS) release comes out every two or three years.
- The current LTS versions are 11 and 17, with Java 8 still being supported for now.
- With its focus on backward compatibility, making changes to Java can often be difficult. Changes restricted to just the library or compiler are often much simpler than changes that also require updates in the virtual machine.
- Java 11 introduced many useful features that are worth upgrading for:
  - The `var` keyword to streamline variable definitions
  - Factory methods to simplify creating lists, maps, and other collections
  - A new `HttpClient` implementation with full HTTP/2 support
  - Single-file programs that can be run directly without compiling to class files

Previous chapter

< Part 1. From 8 to 11 and beyond!

Next chapter

2 Java modules >